

Accessibility Assessment Scheme Tool (AAST)

Frontend Developer Guide

NEXT.JS · REACT 19 · TAILWIND CSS · TANSTACK QUERY

Version: 1.0 | **Release:** 2026-06-17 | **Document ID:** AAST-DOC-FRONTEND-001

© 2026 AAST, Internal — Frontend Engineering Team.

1 EXECUTIVE SUMMARY

The AAST Frontend (the "Client Shell") is a modern, responsive PWA built with **Next.js 16 (App Router)** and **React 19**. It is a **pure presentation layer with zero business logic**.

Core Principle: Zero Business Logic

✔ FRONTEND DOES	✗ FRONTEND NEVER DOES
Render UI components & API responses	Calculations, formulas, or business rules
Send user inputs via REST APIs	Hardcoded thresholds, weights, or scoring
Manage client-side navigation & state	Evaluate or interpret assessment results

Why? The backend is the single source of truth. Any business rule change requires only a database row update — never a frontend redeployment.

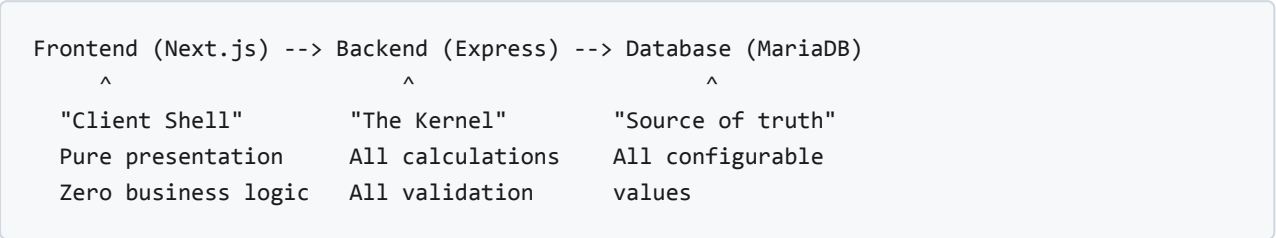
COMPONENT	TECHNOLOGY
Framework	Next.js 16.2.6 (App Router)
UI Library	React 19.2.6
Server State	TanStack React Query 5.x
HTTP Client	Axios 1.5
Styling	Tailwind CSS 4.x
Animation	Framer Motion 12.x
Validation	Zod 3.25
PWA	Service Worker + Web Manifest

Key Features

Assessment dashboard with interactive OBS gauge, searchable criteria grid (63 criteria), building management, certification view, PWA with offline indicator, responsive mobile-first design with bottom navigation, error boundary + Axios interceptors.

2 PROJECT OVERVIEW

Architecture



Page Routes (14 total)

ROUTE	PURPOSE
/	Home — quick links to all sections
/dashboard	Assessment UI with OBS gauge
/criteria	Searchable criteria grid
/certification	Filterable certification results
/reports	Historical evaluation reports
/buildings	Building list with status
/buildings/[id]	Building detail with evaluations
/building	Building type management
/assessments/new	Create assessment form
/assessments/[id]	View saved assessment
/dimensions	Assessment dimension management
/disability	Disability type management
/login	Authentication page

3 FRONTEND OVERVIEW

Folder Structure

```
frontend/
├─ package.json / next.config.js / tsconfig.json
├─ tailwind.config.ts / postcss.config.js
├─ app/                                # App Router pages (14 routes)
│  ├─ layout.tsx                       # Root layout (Inter font, dark theme, PWA)
│  ├─ page.tsx                         # Home page
│  ├─ globals.css                     # Global styles + Tailwind directives
│  ├─ dashboard/page.tsx              # OBS gauge dashboard
│  ├─ criteria/page.tsx               # Criteria grid
│  ├─ certification/page.tsx          # Certification results
│  ├─ reports/page.tsx                # History view
│  ├─ buildings/
│  │  ├─ page.tsx                     # Building list
│  │  └─ [id]/page.tsx                # Building detail
│  ├─ assessments/
│  │  ├─ new/page.tsx                 # Create assessment
│  │  └─ [id]/page.tsx                # Assessment detail
│  ├─ building/page.tsx               # Building types
│  ├─ dimensions/page.tsx             # Dimensions
│  ├─ disability/page.tsx             # Disability types
│  └─ login/page.tsx                  # Auth page
├─ components/
│  ├─ Header.tsx                      # Top navigation
│  ├─ BottomNav.tsx                   # Mobile bottom tabs (5 tabs)
│  ├─ BuildingCard.tsx                # Building summary card
│  ├─ ErrorBoundary.tsx               # React error boundary
│  ├─ OfflineIndicator.tsx            # PWA offline status banner
│  ├─ PWAProvider.tsx                 # Service worker registration
│  └─ layout/
│     ├─ mobile-nav.tsx               # Collapsible mobile drawer
│     └─ navbar.tsx                   # Desktop nav bar
├─ lib/
│  ├─ api/
│  │  ├─ client.ts                    # Axios instance + interceptors
│  │  └─ endpoints.ts                 # 16 typed API wrappers
│  ├─ hooks/
│  │  ├─ useAssessment.ts             # TanStack Query: single assessment
│  │  ├─ useBuildings.ts              # TanStack Query: buildings list
│  │  └─ useCriteria.ts               # TanStack Query: criteria list
│  ├─ queryClient.ts                  # TanStack QueryClient config
│  ├─ animations.ts                   # Shared Framer Motion variants
│  └─ pwa.ts                           # Service worker registration
└─ public/
```

```
|— manifest.json      # PWA manifest
|— sw.js              # Service worker
```

State Management

- **Server State:** TanStack React Query 5 — fetching, caching, refetching
- **Client State:** React `useState` for UI-local state (forms, toggles, modals)
- **Persistence:** `localStorage` under key `accessibility-assessment-state`

Hooks

HOOK	QUERY KEY	PURPOSE
<code>useAssessment(id)</code>	<code>['assessment', id]</code>	Fetch single assessment
<code>useBuildings()</code>	<code>['buildings']</code>	Fetch all buildings with evaluations
<code>useCriteria()</code>	<code>['criteria']</code>	Fetch all 63 criteria

4 API CONSUMPTION

Axios Client (`client.ts`)

```
const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL || "http://localhost:4000/api/v1",
  timeout: 15000,
  headers: { "Content-Type": "application/json" },
});

// Response interceptor normalizes errors into { status, ok, error, message }
api.interceptors.response.use(
  (response) => response,
  (error) => Promise.reject({
    status: error.response?.status || 500,
    ok: false,
    error: error.response?.data?.error || "Network error",
    message: error.message,
  })
);
```

Endpoints (`endpoints.ts`) — 16 typed wrappers

```
export const endpoints = {
  evaluate: (payload) => unwrap<EvaluationResult>(api.post('/evaluate', payload)),
  getCriteria: () => unwrap<Criterion[]>(api.get('/criteria')),
  createCriterion: (p) => unwrap<Criterion>(api.post('/criteria', p)),
  getBuildingTypes: () => unwrap<BuildingType[]>(api.get('/building-types')),
  getNebThresholds: () => unwrap<NebThreshold[]>(api.get('/neb-thresholds')),
  getConfig: () => unwrap<ConfigItem[]>(api.get('/config')),
  getDisabilityTypes: () => unwrap<Label[]>(api.get('/disability-types')),
  getAssessmentDimensions: () => unwrap<Label[]>(api.get('/assessment-dimensions')),
  getBuildings: () => unwrap<Building[]>(api.get('/buildings')),
  getBuilding: (id) => unwrap<Building>(api.get(`/buildings/${id}`)),
  createAssessment: (p) => unwrap<Assessment>(api.post('/assessments', p)),
  getAssessment: (id) => unwrap<Assessment>(api.get(`/assessments/${id}`)),
  getReports: () => unwrap<Report[]>(api.get('/reports')),
  login: (p) => unwrap<AuthResult>(api.post('/auth/login', p)),
};
```

Using Endpoints

```
// Direct call (event handler)
const result = await endpoints.evaluate({ buildingType, scores });

// Via TanStack Query (recommended)
const { data, isLoading, error } = useQuery({
  queryKey: ['criteria'],
  queryFn: () => endpoints.getCriteria(),
});
```

TanStack Query Config

```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 5 * 60 * 1000,    // 5 min
      retry: 1,
      refetchOnWindowFocus: false,
    },
  },
});
```

5 REQUEST LIFECYCLE

```
User -> Next.js Page -> TanStack Query Hook -> Axios Client -> Backend Express
                                         |
User <- React Re-render <- Hook updates cache <- Axios unwrap <- JSON response
```

Step-by-step:

1. User interacts (selects building type, rates criteria, clicks "Evaluate")
 2. Page component collects form state, calls hook mutation
 3. TanStack Query hook calls `endpoints.evaluate()` via Axios
 4. Axios sends POST to `NEXT_PUBLIC_API_URL` with 15s timeout, JSON headers
 5. Response interceptor normalizes into `{ status, ok, error, message }`
 6. Hook updates cache, triggers React re-render
 7. User sees OBS gauge, NEB class, strengths, weaknesses
-

6 LOCAL DEVELOPMENT SETUP

Prerequisites

TOOL	MIN VERSION	CHECK
Node.js	18 LTS (20 rec.)	<code>node --version</code>
npm	9+	<code>npm --version</code>

Backend must be running. See Backend Developer Guide for backend setup.

Quick Start

```
# 1. Install
git clone <repo-url> && cd aast-project/frontend && npm install

# 2. Configure (optional – defaults to localhost:4000)
echo "NEXT_PUBLIC_API_URL=http://localhost:4000/api/v1" > .env.local

# 3. Ensure backend is running
cd ../backend && docker compose up -d mariadb && npx ts-node src/scripts/seed.ts && npm
start

# 4. Start frontend
cd ../frontend && npm run dev
# Output: ▲ Next.js 16.2.6 – Local: http://localhost:3000

# 5. Verify – open http://localhost:3000, navigate to Dashboard, run an evaluation
```

Environment Variables

VARIABLE	DEFAULT
<code>NEXT_PUBLIC_API_URL</code>	<code>http://localhost:4000/api/v1</code>

NPM Scripts

SCRIPT	PURPOSE
<code>npm run dev</code>	Dev server with HMR + Fast Refresh
<code>npm run build</code>	Production build (optimized bundles)
<code>npm start</code>	Production server on port 3000
<code>npm run lint</code>	ESLint

Setup Troubleshooting

SYMPTOM	FIX
"Network error"	Start backend on port 4000
CORS errors	Check <code>NEXT_PUBLIC_API_URL</code>
Port 3000 in use	Kill other dev servers
White screen	Check browser Console (F12)
Styles broken	Check <code>postcss.config.js</code> + <code>tailwind.config.ts</code>

7 COMMON WORKFLOWS

1. Running an Assessment (User Flow)

Open Dashboard → select building type → rate criteria 1–5 → click "Evaluate" → review OBS gauge, NEB class, strengths/weaknesses → optionally save.

2. Adding a New Page

Create `app/feature/page.tsx` → add TanStack Query hook if needed → add endpoint to `endpoints.ts` if needed → add nav link in `Header.tsx` or `BottomNav.tsx` → test with `npm run dev`.

3. Adding a New API Endpoint

```
// lib/api/endpoints.ts
getNewResource: () => unwrap<NewType[]>(api.get('/new-resource')),
```

4. Creating a Reusable Component

Create `components/MyComponent.tsx` → add `"use client"` if interactive → export default → import in pages.

5. Styling with Tailwind

```
<div className="flex items-center gap-4 p-6 bg-white rounded-lg shadow-md">
  <h2 className="text-xl font-semibold text-gray-900">Title</h2>
</div>
```

Mobile-first responsive, dark mode via Tailwind classes, shared variants in `lib/animations.ts`.

6. Debugging API Calls

Open DevTools (F12) → Network tab → filter by `/api/v1` → inspect request headers, payload, response. Check Console for Axios interceptor errors.

7. PWA Testing

Use Lighthouse PWA audit. Verify `manifest.json` is valid, `sw.js` is registered, site served over HTTPS or localhost.

8 DEBUGGING GUIDE

Top 5 Errors

1. "Network Error" in Browser

Fix: Backend not running — start backend. Check `NEXT_PUBLIC_API_URL`. Ensure `cors()` middleware is active.

2. "Hydration Error" in Console

Fix: Server/client rendered different HTML. Use `"use client"` or `useEffect` for browser-only code (`window`, `document`).

3. "useSearchParams() should be wrapped in Suspense"

Fix: Wrap component in `<Suspense fallback={<div>Loading...</div>}>`.

4. White Screen / Blank Page

Fix: Check browser Console for JS errors, Network tab for failed API calls, verify `ErrorBoundary` is wrapping the tree.

5. TanStack Query Stale Data

Fix: Use `queryClient.invalidateQueries()` after mutations, check `staleTime` in `queryClient.ts`, force refetch with `refetch()`.

Log Locations

COMPONENT	HOW TO VIEW
Frontend Console	F12 → Console tab
Network Requests	F12 → Network tab
React Components	React DevTools extension
TanStack Query	Built-in devtools (dev only)
Build Errors	Terminal where <code>npm run dev</code> runs

9 TROUBLESHOOTING

#	PROBLEM	LIKELY FIX
1	Can't reach API	Check <code>NEXT_PUBLIC_API_URL</code> , verify backend running
2	CORS errors	Ensure backend <code>cors()</code> middleware; check origin
3	White screen	Check browser Console for render errors
4	PWA not installing	Validate <code>manifest.json</code> + <code>sw.js</code> ; use Lighthouse
5	Hydration mismatch	Add <code>"use client"</code> or <code>useEffect</code> for browser-only code
6	Styles broken	Check postcss + tailwind config; restart dev server
7	Build fails	<code>npm ci</code> then <code>npm run build</code> ; check Next.js version
8	Slow page load	Audit bundle size + API response time; use Lighthouse
9	Infinite re-render	Check <code>useEffect</code> dependencies; use React DevTools profiler
10	Form state lost	Persist to <code>localStorage</code> or query params

10 CODING STANDARDS

Core Principles

1. **Zero Business Logic:** Never implement calculations, formulas, or thresholds in the frontend
2. **TypeScript Strict:** Explicit types, no `any`, interfaces for API responses
3. **Server vs Client:** Minimize `"use client"` — prefer Server Components
4. **Component Composition:** Small, focused, single-responsibility components

Naming Conventions

ELEMENT	CONVENTION	EXAMPLE
Components	PascalCase	<code>BuildingCard</code>
Hooks	camelCase, <code>use</code> prefix	<code>useBuildings</code>
Functions/variables	camelCase	<code>fetchBuilding()</code>
Types/Interfaces	PascalCase	<code>EvaluationResult</code>

Component Rules

- Functional components only (no classes)
- `"use client"` only when using `useState`, `useEffect`, event handlers, browser APIs, or TanStack Query
- Use TanStack Query hooks for server data (never fetch directly in components)
- Use Framer Motion `motion.*` for animations
- Export default for pages, named for shared components

Prohibited Patterns

```
// ❌ PROHIBITED – business logic in frontend
const obs = (rawScore / 25) * 100;
const nebClass = obs >= 85 ? 'A+' : 'A';
const weights = [0.2, 0.2, 0.2, 0.2, 0.2];

// ✅ CORRECT – pass-through to backend
const result = await endpoints.evaluate({ buildingType, scores });
const { obs, nebClass } = result;
```

Formatting: 2-space indent, no trailing whitespace, files end with newline, semicolons required.

11 GIT WORKFLOW

Branch strategy: `main` (production) ← `develop` (integration) ← `feature/*` / `fix/*` / `chore/*`

Commit format: `<type>(frontend): <description>` — types: `feat`, `fix`, `chore`, `docs`, `test`, `refactor`

PR process: Create feature branch → commit → `npm run build && npm run lint` → push → PR to `develop` → CI runs `build-frontend` → code review → merge.

CI pipeline (on push to main): test-backend → build-backend → deploy-backend → build-frontend → deploy-frontend (5 jobs).

12 DEPLOYMENT

Azure Static Web App (Standard tier)

- **Build:** `next build` produces optimized static + dynamic output
- **Hosting:** Global CDN with automatic SSL
- **Env Vars:** Set in Azure Portal → Static Web App → Configuration

```
# Production env var
NEXT_PUBLIC_API_URL=https://app-aas-backend-production.azurewebsites.net/api/v1
```

Post-Deployment Verification

1. Open production URL, verify all pages load
 2. Check Dashboard connects to production backend
 3. Run test evaluation — verify OBS gauge renders
 4. Test mobile responsiveness
 5. Run Lighthouse audit (performance, accessibility, PWA)
-

13 FAQ

Q: Why zero business logic? A: Single source of truth in the backend. Business rule changes never require frontend redeployment.

Q: Why Next.js App Router instead of Pages Router? A: React Server Components, streaming, improved layouts — recommended for new Next.js projects.

Q: Why TanStack Query instead of Redux? A: Specializes in server state (fetch/cache/sync). Minimal client-only state means full state management is overkill.

Q: When to use `"use client"`? A: When using `useState`, `useEffect`, event handlers, browser APIs, or TanStack Query hooks.

Q: How to handle loading/error states? A: TanStack Query provides `isLoading`, `isFetching`, `isError`, `error`. Show spinners/skeletons for loading, fallback UI for errors.

Q: How to change API URL per environment? A: Set `NEXT_PUBLIC_API_URL` in `.env.local` (dev) or Azure Portal (prod).

14 USEFUL COMMANDS

COMMAND	PURPOSE
<code>npm run dev</code>	Start dev server (HMR)
<code>npm run build</code>	Production build
<code>npm start</code>	Production server
<code>npm run lint</code>	ESLint
<code>curl localhost:4000/health</code>	Backend health check
<code>curl localhost:4000/api/v1/criteria</code>	Get all criteria
<code>npx next info</code>	System debug info
F12	Open browser DevTools
Ctrl+Shift+E	Network tab
Ctrl+Shift+J	Console tab

15 GLOSSARY

TERM	DEFINITION
AAST	Accessibility Assessment Scheme Tool
App Router	Next.js 13+ file-based routing via <code>app/</code> directory
Axios	Promise-based HTTP client
Client Component	React component with <code>"use client"</code> — runs in browser
Server Component	React component rendered on server (default in App Router)
Framer Motion	React animation library
HMR	Hot Module Replacement — instant updates without full reload
NEB	National Evaluation Benchmark (A+, A, B, No Rating)
OBS	Overall Building Score (0–100%)
PWA	Progressive Web App — installable, offline-capable
TanStack Query	Server state management (fetching, caching, sync)
Tailwind CSS	Utility-first CSS framework
Zod	TypeScript-first schema validation
Service Worker	Background script for offline support & caching
Hydration	Attaching React event listeners to server-rendered HTML

Accessibility Assessment Scheme Tool (AAST)

Backend Developer Guide

EXPRESS · TYPESCRIPT · MARIADB · EN 17210

Version: 1.0 | **Release:** 2026-06-17 | **Document ID:** AAST-DOC-BACKEND-001

© 2026 AAST. Internal — Backend Engineering Team.

1 EXECUTIVE SUMMARY

The AAST Backend (the "Kernel") is a metadata-driven REST API implementing the **EN 17210** accessibility scoring framework. It is the single source of truth for all business logic, calculations, and data access.

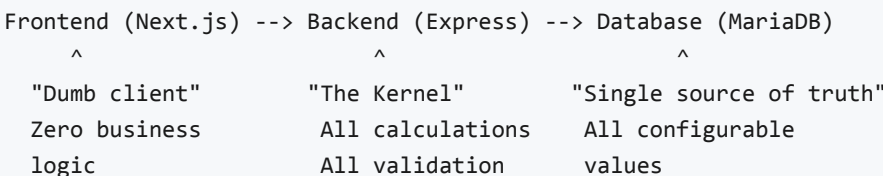
RESPONSIBILITY	IMPLEMENTATION
Calculation Engine	5-layer EN 17210 algorithm (IS → TIS → CIS → OBS → NEB)
Data Persistence	TypeORM + MariaDB with 5 entities
REST API	14 endpoints across 6 controllers
Validation	Zod schemas for all POST/PUT
Caching	Lazy-initialized in-memory metadata cache
Security	Helmet, CORS, HTTPS, TLS 1.2
Enforcement	Static test suite bans hardcoded patterns

Core principle: All configurable values live in the database — weights, thresholds, criteria, and configuration. Domain experts update the system via SQL or API. No code changes. No deployments.

COMPONENT	TECHNOLOGY
Runtime	Node.js 20 LTS
Framework	Express.js 4.18
Language	TypeScript 5.1+ (ES2022, CommonJS)
ORM	TypeORM 0.3.17
Database	MariaDB 10.11
Validation	Zod 4.4+
Containerization	Docker Compose
Cloud	Azure (Bicep IaC)
CI/CD	GitHub Actions

2 PROJECT OVERVIEW

Architecture



Key Architectural Decisions

1. **Metadata-Driven:** All configurable values in the database, never in code
2. **Single Source of Truth:** Backend is sole authority for calculations
3. **Dynamic Type Detection:** DT 1–5 and AD 1–5 derived from criteria data
4. **Config-Driven Thresholds:** NEB breakpoints, strength/weakness, MAX_OBS are all DB rows
5. **Lazy Metadata Caching:** Initializes once, caches lookup data in memory
6. **Static Test Enforcement:** CI blocks deployment if hardcoded patterns detected

Business Purpose

The AAST evaluates buildings against **63 canonical criteria** across a 5×5 matrix:

- **5 Disability Types (DT):** Physical, Sensory, Cognitive & Neurodiverse, Communication & Mental Health, Multiple/Situational
- **5 Assessment Dimensions (AD):** Spatial & Physical, Safety & Environmental, Cognitive & Navigational, Digital Interaction, Social Inclusion

Each criterion has a unique code `EC{DT}.{AD}.{index}` (e.g., `EC1.1.1`).

Certification flow: Assessor selects building type → rates criteria 1–5 → Backend computes weighted OBS% → Maps to NEB class (A+/A/B/No Rating) → Returns strengths, weaknesses, per-DT/AD scores → Persists with full audit trail.

Targets compliance with **EN 17210** and EU accessibility regulations.

3 SYSTEM ARCHITECTURE

```
+-----+
|                PRESENTATION LAYER                |
| Next.js Pages --> React Components --> Custom Hooks --> Axios |
+-----+
|                API GATEWAY                        |
| Express Router -> Helmet -> CORS -> JSON Parser -> Error Handler |
+-----+
|                BUSINESS LOGIC LAYER              |
| Calculation Service (5-layer engine) + Metadata Service |
| Zod Validation                                         |
+-----+
|                DATA ACCESS LAYER                |
| TypeORM Repositories --> Entities --> Migrations      |
+-----+
|                PERSISTENCE LAYER                 |
|                MariaDB 10.11                     |
+-----+
```

Data flow: User Input → Hook → Axios POST → Router → Controller → Service → Repository → MariaDB → Response → React Render

4 BACKEND OVERVIEW

Folder Structure

```
backend/
├─ docker-compose.yml          # MariaDB container
├─ docker-initdb.d/init.sql    # DB user/database creation
├─ package.json / tsconfig.json
├─ src/
│   ├─ index.ts                # Express entry point
│   ├─ data-source.ts          # TypeORM DataSource config
│   └─ controllers/            # 6 route handlers
│       ├─ evaluateController.ts # POST /api/v1/evaluate
│       ├─ dataController.ts    # GET/POST criteria, types, thresholds, config
│       ├─ buildingsController.ts # GET buildings
│       ├─ assessmentsController.ts # POST/GET assessments
│       ├─ reportsController.ts  # GET reports
│       └─ authController.ts     # POST /api/v1/auth/login
│   └─ services/
│       ├─ calculationService.ts # 5-layer EN 17210 engine
│       └─ metadataService.ts    # Lazy cache + validation
│   └─ entities/                # Criterion, BuildingType, Evaluation, NebThreshold,
Config
├─ migrations/                  # 3 sequential schema migrations
├─ data/                        # Seed data (criteria.ts, building-types.json)
├─ scripts/                     # seed.ts, runMigrations.ts, workbook scripts
└─ tests/                       # calculation.test.ts, static.test.ts, metadata.test.ts
```

Controllers

Controllers handle request parsing, validation, and response formatting. **No business logic** — they delegate to services.

CONTROLLER	KEY ENDPOINTS	PURPOSE
<code>evaluateController</code>	<code>POST /evaluate</code>	Full EN 17210 evaluation
<code>dataController</code>	<code>GET/POST /criteria</code> , <code>GET /building-types</code> , <code>GET /neb-thresholds</code> , <code>GET /config</code>	Dynamic data CRUD
<code>buildingsController</code>	<code>GET /buildings</code> , <code>GET /buildings/:id</code>	Building directory
<code>assessmentsController</code>	<code>POST /assessments</code> , <code>GET /assessments/:id</code>	Assessment persistence
<code>reportsController</code>	<code>GET /reports</code>	Evaluation history
<code>authController</code>	<code>POST /auth/login</code>	Mock JWT login

Services

Calculation Service — Implements the 5-layer scoring algorithm (see Section 6).

Metadata Service — Lazy-initialized cache:

- Initializes DataSource connection once
- Caches NEB thresholds (ordered DESC by `min`)
- Caches config values per key
- Provides `getBuildingType()`, `getAllCriteria()`, `validateMetadata()`
- Validates all 4 core tables have data on startup; exits code 1 if not

Static Test Enforcement

`static.test.ts` greps ALL source files for banned hardcoded patterns:

- ❌ `0.2,0.2,0.2,0.2,0.2` — equal weights must be DB-driven
- ❌ `85, 60, 40` — NEB thresholds must be in `neb_thresholds` table
- ❌ `MAX_OBS = 25` — config must be in `config` table

This runs in CI and blocks deployment if violated.

5 DATABASE OVERVIEW

Entity-Relationship

```
erDiagram
    building_types {
        VARCHAR_36 id PK
        VARCHAR_255 name UK
        JSON disability_weights
        JSON dimension_weights
    }
    criteria {
        VARCHAR_36 id PK
        VARCHAR_255 code UK
        TEXT name definition justification
        FLOAT value
        INT disability dimension
        INT score
        JSON levels
    }
    evaluations {
        VARCHAR_36 id PK
        VARCHAR_255 user_id building_type
        JSON scores result
        DATETIME created_at
        DOUBLE obs
        VARCHAR neb_class
        DOUBLE average_raw_score
    }
    neb_thresholds {
        VARCHAR_36 id PK
        INT min
        VARCHAR_50 neb_class
        VARCHAR_50 equivalent
        TEXT meaning
        INT neb_score ordinal
    }
    config {
        VARCHAR_36 id PK
        VARCHAR_255 key UK
        TEXT value
    }
    evaluations ||--o{ building_types : "building_type refs name"
```

Table Details

building_types (7 rows)

Stores per-type weight matrices for DT and AD.

COLUMN	TYPE	DESCRIPTION
name	VARCHAR(255) UNIQUE	e.g., "Commercial Buildings"
disability_weights	JSON	5 floats for DT 1–5
dimension_weights	JSON	5 floats for AD 1–5

Example: { "name": "Commercial Buildings", "disability_weights": [0.28,0.22,0.18,0.12,0.2], "dimension_weights": [0.30,0.25,0.18,0.12,0.15] }

criteria (63 rows)

Canonical evaluation criteria. Each has value (importance weight 0–1), disability and dimension classification (1–5), default score (3), and levels (array of 5 descriptors). Example: EC1.1.1 — "Accessible Entrance Width" — value 0.7, DT 1, AD 1.

evaluations (variable rows)

One row per assessment. Stores scores (code→score map), computed result (full JSON), obs (%), neb_class, average_raw_score.

neb_thresholds (4 rows)

MIN	NEB_CLASS	MEANING
85	A+	Excellent / Best practice
60	A	Good
40	B	Acceptable
0	No Rating	Non-compliant

`config` (5 rows)

KEY	VALUE	DESCRIPTION
<code>MAX_OBS</code>	25	Maximum OBS raw score
<code>DEFAULT_SCORE</code>	3	Fallback score
<code>STRENGTH_THRESHOLD</code>	5	Min score for strength
<code>WEAKNESS_THRESHOLD</code>	2	Max score for weakness
<code>TOP_N_RESULTS</code>	5	Results to return

6 CALCULATION ENGINE

Implements the **EN 17210 accessibility scoring framework** — purely metadata-driven, no hardcoded formulas.

The 5-Layer Algorithm

Layer 1: IS 5: NEB Impact Score Classification per criterion IS = value x score threshold OBS >= min	Layer 2: TIS Total Impact Score per DT TIS = sum IS per DT	Layer 3: CIS Combined Impact Score per AD CIS = sum(dtW x IS) per AD	Layer 4: OBS Overall Building Score % OBS = sum(dtWxTIS) /MAX_OBS x 100	Layer Lookup Find 1st where
--	--	---	--	---

Layer 1: IS (Impact Score)

$IS(c) = c.value \times c.score$ — rounded to 4 decimal places.

Layer 2: TIS (Total Impact Score)

$TIS(dt) = \sum IS(c)$ for all criteria where $c.disability == dt$. DT IDs are **derived from data** — not hardcoded.

Layer 3: CIS (Combined Impact Score)

$CIS(ad) = \sum (dtWeight[dt] \times TIS(dt))$ for all criteria where $c.dimension == ad$.

Layer 4: OBS (Overall Building Score)

$OBS_{raw} = \sum (dtWeight[dt] \times TIS(dt))$ for all DTs. $OBS_{pct} = (OBS_{raw} / MAX_OBS) \times 100$. MAX_OBS is read from `config` table (value: 25). Rounded to 2 decimal places.

Layer 5: NEB Classification

Thresholds loaded DESC by `min`: 85, 60, 40, 0. Find first where $OBS\% \geq threshold.min \rightarrow A+ (\geq 85)$, A (≥ 60), B (≥ 40), No Rating (< 40).

Additional Computations

- **Strengths/Weaknesses:** Top N where score ≥ STRENGTH_THRESHOLD or ≤ WEAKNESS_THRESHOLD (from config)
- **Per-DT/AD averages:** Arithmetic mean of scores grouped by DT/AD

Metadata-Driven Design

Every configurable value lives in the database, never in code:

HARDCODED (X)	METADATA-DRIVEN (✓)
<code>const NEB = [85, 60, 40]</code>	<code>SELECT * FROM neb_thresholds ORDER BY min DESC</code>
<code>const MAX_OBS = 25</code>	<code>SELECT value FROM config WHERE key='MAX_OBS'</code>
Fixed DT/AD arrays	Derived from <code>criteria.disability</code> / <code>.dimension</code>
Redeploy to change	Update DB row — instant effect

Key guarantees: DT/AD sets derived from data; weights per building type from DB; NEB thresholds are DB rows; MAX_OBS is configurable; static tests enforce compliance.

7 API ARCHITECTURE

- **Base URL:** `/api/v1`
- **Response Envelope:** `{ ok: boolean, data?: any, error?: string }`
- **Status Codes:** 200, 201, 400, 404, 409, 500

Endpoint Reference

Evaluation

METHOD	PATH	AUTH	DESCRIPTION
POST	<code>/evaluate</code>	No	Full EN 17210 computation

Request: `{ "buildingType": "Commercial Buildings", "scores": { "EC1.1.1": 5 } }`

Response: `{ ok: true, result: { obs, nebClass, nebMeaning, nebScore, averageRawScore, avgRawScoreByDT, avgRawScoreByAD, strengths, weaknesses, tisByDT, cisByAD, criteria } }`

Data CRUD

METHOD	PATH	DESCRIPTION
GET	<code>/criteria</code>	All 63 criteria
GET	<code>/criteria/:code</code>	Single criterion
POST	<code>/criteria</code>	Add criterion dynamically
GET	<code>/building-types</code>	7 building types with weights
GET	<code>/neb-thresholds</code>	NEB classification thresholds
GET	<code>/config</code>	All config key-values
GET	<code>/disability-types</code>	5 DT labels
GET	<code>/assessment-dimensions</code>	5 AD labels

Assessments & Reports

METHOD	PATH	DESCRIPTION
GET	<code>/buildings</code>	All buildings (joined with evaluations)
GET	<code>/buildings/:id</code>	Building detail
POST	<code>/assessments</code>	Create & save assessment
GET	<code>/assessments/:id</code>	Retrieve saved assessment
GET	<code>/reports</code>	Evaluation history

Auth

METHOD	PATH	DESCRIPTION
POST	<code>/auth/login</code>	Mock JWT login

Dynamic Criteria System

Adding a criterion is a single API call or SQL INSERT — no code changes, no deployment:

```
curl -X POST http://localhost:4000/api/v1/criteria \
  -H "Content-Type: application/json" \
  -d
'{"code":"EC1.1.10","name":"...", "definition":"...", "value":0.15, "disability":1, "dimension":1, "levels":["L1", "L2", "L3", "L4", "L5"]}'
```

Validation rules: `code` must match `/^EC\d+\.\d+\.\d+$/`, `value` 0–1, `disability` / `dimension` 1–5, `levels` exactly 5 strings, duplicate code → 409. The next evaluation automatically includes the new criterion.

8 REQUEST LIFECYCLE

Evaluation Request Flow

```
Browser -> Next.js Hook -> Axios POST /api/v1/evaluate
  -> Express: Helmet -> CORS -> JSON parse
  -> evaluateController: validates body shape
    -> calculationService.evaluate(buildingType, scores)
      -> Read MAX_OBS, thresholds, weights from metadata cache
      -> Load all 63 criteria, apply scores
      -> Compute IS -> TIS -> CIS -> OBS -> NEB
      -> Compute strengths, weaknesses, averages
      -> Return result object
    -> Controller: res.json({ ok: true, result })
  -> Express: 200 JSON response
  -> Axios: unwrap result
  -> React: re-render with OBS gauge, NEB class, strengths, weaknesses
```

Controller Validation

`evaluateController.ts` checks `buildingType` is string/undefined and `scores` is object/undefined. Returns 400 with specific message on failure.

Response Object Shape

`obs`, `nebClass`, `nebEquivalent`, `nebMeaning`, `nebScore`, `averageRawScore`, `avgRawScoreByDT`, `avgRawScoreByAD`, `strengths` (top 5), `weaknesses` (top 5), `tisByDT`, `cisByAD`, `criteria` (all 63 with computed IS).

9 LOCAL DEVELOPMENT SETUP

Prerequisites

TOOL	MIN VERSION	CHECK
Node.js	18 LTS (20 rec.)	<code>node --version</code>
npm	9+	<code>npm --version</code>
Docker	24+	<code>docker --version</code>

Quick Start

```
# 1. Clone and install
git clone <repo-url> && cd aast-project/backend && npm install

# 2. Start MariaDB
docker compose up -d mariadb
# Wait ~10s, verify: docker compose ps

# 3. Seed database
npx ts-node src/scripts/seed.ts
# Output: "DataSource initialized" / "Seeding complete"

# 4. Start backend
npm start
# Output: "Backend listening on 4000"

# 5. Verify
curl http://localhost:4000/health          # -> {"status":"ok"}
curl http://localhost:4000/api/v1/criteria # -> 63 criteria
```

Docker Environment

MariaDB only runs in Docker; backend/frontend run natively.

PROPERTY	VALUE
Image	<code>mariadb:10.11</code>
Host Port	<code>3306</code>
Database	<code>mydb</code>
User/Pass	<code>myuser</code> / <code>mypassword</code>
Root Pass	<code>rootpass</code>
Data Volume	<code>db_data</code>

Environment Variables

VARIABLE	DEFAULT
<code>PORT</code>	<code>4000</code>
<code>DATABASE_URL</code>	<code>mysql://myuser:mypassword@127.0.0.1:3306/mydb</code>
<code>DB_TYPE</code>	<code>mariadb</code>
<code>TYPEORM_SYNCHRONIZE</code>	<code>false</code>

NPM Scripts

SCRIPT	PURPOSE
<code>npm start</code>	Start backend
<code>npm test</code>	Run calculation tests
<code>npm run test:automated</code>	Full suite (metadata + static + calculation)
<code>npm run seed</code>	Seed database
<code>npm run migrate</code>	Run migrations
<code>npm run verify:workbook</code>	Verify Excel workbook mapping

Setup Troubleshooting

SYMPTOM	FIX
<code>ECONNREFUSED :3306</code>	<code>docker compose up -d mariadb</code>
<code>Access denied</code>	Check DATABASE_URL matches docker-compose.yml
<code>Table doesn't exist</code>	<code>npx ts-node src/scripts/seed.ts</code>
<code>Metadata validation failed</code>	Run seed script
<code>Port 4000 in use</code>	<code>netstat -ano findstr 4000</code> then kill PID

10 RUNNING THE PROJECT

Manual Startup (Windows)

Terminal 1 — MariaDB: `cd backend && docker compose up -d mariadb`

Terminal 2 — Backend: `cd backend && npm start`

Terminal 3 — Frontend (optional): `cd frontend && npm run dev`

Dev Shell Scripts

SCRIPT	PURPOSE
<code>start-dev.sh</code>	Start all services
<code>stop-dev.sh</code>	Stop all services
<code>status-dev.sh</code>	Check service status
<code>logs-dev.sh</code>	View service logs

Quick Verification

```
curl localhost:4000/health    # -> {"status":"ok"}

curl -X POST localhost:4000/api/v1/evaluate \
  -H "Content-Type: application/json" \
  -d '{"buildingType":"Commercial Buildings"}'    # -> full result JSON
```

11 COMMON WORKFLOWS

1. Add a New Criterion

```
curl -X POST localhost:4000/api/v1/criteria -H "Content-Type: application/json" \
-d
'{"code":"EC1.1.10","name":"...", "definition":"...", "value":0.15, "disability":1, "dimension":1, "levels":["L1","L2","L3","L4","L5"]}'
```

Or via SQL:

```
INSERT INTO criteria (id, code, name, definition, justification, value, disability,
dimension, score, levels)
VALUES (UUID(), 'EC1.1.10', '...', '...', '...', 0.15, 1, 1, 3,
'["L1","L2","L3","L4","L5"]');
```

2. Adjust NEB Thresholds

```
UPDATE neb_thresholds SET min = 65 WHERE neb_class = 'A';
INSERT INTO neb_thresholds VALUES (UUID(), 95, 'A++', '5+', 'Outstanding', 5, 0);
```

3. Create a Building Type

```
INSERT INTO building_types VALUES (
  UUID(), 'Healthcare Buildings', '[0.20,0.25,0.20,0.20,0.15]',
  '[0.30,0.30,0.15,0.10,0.15]'
);
```

4. Update Config Values

```
UPDATE config SET value = '30' WHERE `key` = 'MAX_OBS';
UPDATE config SET value = '4' WHERE `key` = 'STRENGTH_THRESHOLD';
```

5. Run Tests Before Commit

```
npm run test:automated
# Expected: "Metadata validation passed" / "Static checks passed" / "All tests passed"
```

6. Debug a Calculation

Send test evaluation via curl → examine `tisByDT`, `cisByAD`, `criteria[0].is` → compare against Excel workbook reference.

7. Database Backup & Restore

```
docker exec mariadb mysqldump -u myuser -pmypassword mydb > backup.sql
docker exec -i mariadb mysql -u myuser -pmypassword mydb < backup.sql
```

8. Reset Database

```
docker compose down -v && docker compose up -d mariadb && npx ts-node src/scripts/seed.ts
```

12 DEBUGGING GUIDE

Top 5 Errors

1. "DataSource init failed"

Causes: MariaDB not running, wrong DATABASE_URL, port conflict. **Fix:** `docker compose up -d mariadb`; verify credentials match docker-compose.yml.

2. "Metadata validation failed" (exit code 1)

Fix: Run `npx ts-node src/scripts/seed.ts`. Check all 4 core tables have data.

3. "ECONNREFUSED 127.0.0.1:4000"

Fix: Backend not running — `npm start`.

4. CORS Errors in Browser

Fix: Backend uses `cors()` middleware. If frontend port changed, update CORS config in `index.ts`.

5. "Missing configuration: MAX_OBS"

Fix: Re-run seed or insert manually:

```
INSERT INTO config VALUES (UUID(), 'MAX_OBS', '25');
```

Log Locations

COMPONENT	HOW TO VIEW
Backend	Terminal where <code>npm start</code> runs (stdout)
MariaDB	<code>docker compose logs mariadb</code>
Docker	<code>docker compose logs</code>

13 TROUBLESHOOTING

#	PROBLEM	LIKELY FIX
1	Backend won't start	<code>docker compose up -d mariadb</code> then <code>npm start</code>
2	"Access denied for user"	Verify DATABASE_URL matches docker-compose.yml
3	Tables don't exist	<code>npx ts-node src/scripts/seed.ts</code>
4	OBS always 0	Check <code>SELECT * FROM config WHERE key='MAX_OBS'</code>
5	NEB always "No Rating"	Re-seed <code>neb_thresholds</code> table
6	Empty strengths/weaknesses	Check STRENGTH_THRESHOLD & WEAKNESS_THRESHOLD in config
7	"entity.parse.failed"	Invalid JSON in request body — validate payload
8	TypeORM "EntityMetadataNotFound"	Verify all 5 entities in <code>data-source.ts</code> entities array
9	Docker pull fails	Check network; <code>docker pull mariadb:10.11</code> manually
10	TypeScript compilation errors	<code>npm install</code> then <code>npx tsc --noEmit</code>
11	Static test fails in CI	Remove hardcoded values; use DB config
12	Data lost after restart	Verify <code>db_data</code> volume exists: <code>docker volume ls</code>

14 CODING STANDARDS

Core Principles

1. **Metadata-Driven First:** Never hardcode a database-worthy value
2. **Backend Owns Logic:** All calculations & business rules in services
3. **TypeScript Strict:** `strict: true`, no implicit `any`
4. **Single Source of Truth:** Canonical seed data in `src/data/`

Naming Conventions

ELEMENT	CONVENTION	EXAMPLE
Files	kebab-case	<code>calculation-service.ts</code>
Classes/Entities	PascalCase	<code>NebThreshold</code>
Functions/variables	camelCase	<code>getBuildingType()</code>
DB tables/columns	snake_case	<code>neb_thresholds</code> , <code>building_type</code>
API paths	kebab-case	<code>/api/v1/building-types</code>

Service & Controller Rules

- Services export **async functions**, never classes
- Services use metadata service for data access (not direct repos)
- Calculation service is the **only** place formulas exist
- Controllers validate shape before calling services
- Controllers use `try/catch` → 400 for validation, 500 for internal errors
- Success responses always include `ok: true`

Prohibited Patterns (CI-enforced)

```
// WRONG - hardcoded
const weights = [0.2, 0.2, 0.2, 0.2, 0.2];
const NEB = [85, 60, 40];
const MAX_OBS = 25;

// CORRECT - from database
const weights = bt.disability_weights;
const neb = await metadata.getNebThresholds();
const MAX_OBS = await metadata.getConfigNumber('MAX_OBS');
```

Formatting: 2-space indent, no trailing whitespace, files end with newline, semicolons required.

15 GIT WORKFLOW

Branch strategy: `main` (production) ← `develop` (integration) ← `feature/*` / `fix/*` / `chore/*`

Commit format: `<type><scope>: <description>` — types: `feat`, `fix`, `chore`, `docs`, `test`, `refactor`

PR process: Create feature branch → commit → `npm run test:automated` → push → PR to `develop` → CI runs → code review → merge.

CI pipeline (on push to main): test-backend → build-backend → deploy-backend → build-frontend → deploy-frontend (5 jobs, GitHub Actions).

16 DEPLOYMENT

Azure Resources (via `main.bicep`)

RESOURCE	SKU
App Service Plan	B1 Basic Linux
Backend App Service	Node 20 LTS
Static Web App	Standard
MariaDB Server	10.3, TLS 1.2
Application Insights + Log Analytics	Monitoring

Deploy

```
az deployment sub create \  
  --location westeurope \  
  --template-file azure/main.bicep \  
  --parameters dbPassword='<password>' dbAdminPassword='<admin>'
```

Production App Settings

```
PORT=4000  
WEBSITES_PORT=4000  
DATABASE_URL=mysql://myuser:<pwd>@mariadb-aas-  
production.mariadb.database.azure.com:3306/mydb  
DB_TYPE=mariadb  
TYPEORM_SYNCHRONIZE=false
```

17 FAQ

Q: Why TypeORM and not Prisma? A: Mature MariaDB support, entity decorators, built-in migration system, and repository pattern.

Q: Why lazy metadata initialization? A: Defers DB connection to first request — server starts even if DB is temporarily unavailable.

Q: Why derive DT/AD from data instead of config? A: System auto-adapts when criteria are added/removed without code changes.

Q: How do I reset the database? A: `docker compose down -v && docker compose up -d mariadb && npx ts-node src/scripts/seed.ts`

Q: Migrations vs seed? A: Migrations = schema (DDL), seed = data (DML). Migrations create tables; seed populates canonical values.

Q: Can I use PostgreSQL? A: Changing `DB_TYPE` to `postgres` and updating the connection string should work but is untested.

18 USEFUL COMMANDS

COMMAND	PURPOSE
<code>docker compose up -d mariadb</code>	Start MariaDB
<code>docker compose down</code>	Stop MariaDB
<code>docker compose logs -f mariadb</code>	Follow DB logs
<code>docker exec -it mariadb mariadb -u myuser -pmypassword mydb</code>	DB CLI
<code>npm start</code>	Start backend
<code>npm run test:automated</code>	Full test suite
<code>npm run seed</code>	Seed database
<code>curl localhost:4000/health</code>	Health check
<code>curl localhost:4000/api/v1/criteria</code>	Get all criteria
<code>curl localhost:4000/api/v1/building-types</code>	Get building types
<code>curl localhost:4000/api/v1/neb-thresholds</code>	Get NEB thresholds
<code>curl localhost:4000/api/v1/config</code>	Get config
<code>SHOW TABLES;</code>	List DB tables
<code>SELECT COUNT(*) FROM criteria;</code>	Should return 63

19 GLOSSARY

TERM	DEFINITION
AAST	Accessibility Assessment Scheme Tool
AD	Assessment Dimension (1–5): Spatial, Safety, Cognitive, Digital, Social
CIS	Combined Impact Score — weighted sum per AD
DT	Disability Type (1–5): Physical, Sensory, Cognitive, Communication, Multiple
EN 17210	European Standard for built environment accessibility
IS	Impact Score = value × score per criterion
TIS	Total Impact Score = Σ IS per DT
OBS	Overall Building Score = weighted percentage (0–100)
NEB	National Evaluation Benchmark (A+, A, B, No Rating)
MAX_OBS	Maximum possible OBS raw value (25, configurable)
TypeORM	TypeScript ORM for Node.js
Zod	TypeScript-first schema validation
Bicep	Azure infrastructure-as-code language
Seed	Initial data population process
Static Test	Test scanning source code for banned hardcoded patterns